
Práctica 3 - Diseño y simulación de un sumador de 4 bits en VHDL

Alejandro Otero Vázquez

13 de enero de 2025

En este documento se detalla la metodología para el diseño e implementación de un sumador binario de 4 bits en lenguaje VHDL.

1. Utilizando sumadores completos

Para describir el sumador de 4 bits una opción puede ser a partir de las funciones lógicas que definan la suma y acarreo binarias para un solo bit. Se este modo y a partir de un bucle for se podrán diseñar 4 sumadores binarios concatenados de modo que el acarreo de salida de un sumador sea el acarreo de entrada del sumador correspondiente al siguiente bit.

Se definen las señales de entrada y salida binarias del siguiente modo:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity sumador_paralelo is
    Port ( A : in  STD_LOGIC_VECTOR (3 downto 0);
          B : in  STD_LOGIC_VECTOR (3 downto 0);
          Cin: in  STD_LOGIC;
          Cout : out  STD_LOGIC;
          S : out  STD_LOGIC_VECTOR (3 downto 0));
end sumador_paralelo;
```

Donde A y B son las entradas de 4 bits y Cin el acarreo de entrada que pudiera tener el sumador si se concatenase con otro para generar un sumador de n-bits. La salida S también será un vector de 4 bits.

1.1. VHDL Module

La arquitectura de la entidad **sumadorparalelo**, como se ha anunciado al principio de esta sección, se hará a partir de un bucle for. Se define además una señal auxiliar de 5 bits para el acarreo, que incluirá Cin y los acarreos de los 4 sumadores.

```
architecture Behavioral of sumador_paralelo is
    signal c: std_logic_vector(4 downto 0) := (others => '0');
begin
    c(0) <= Cin;
    sumador: for i in 0 to 3 generate
        — Full adder —
        c(i+1) <= (A(i) and B(i)) or (A(i) and c(i)) or (B(i) and c(i));
        S(i) <= A(i) xor B(i) xor c(i);
    end generate;
    Cout <= c(4);
end Behavioral;
```

1.2. VHDL Test Bench

En el archivo del test bench se especifican los valores que le daremos a las señales de entrada durante la simulación. Como tenemos 9 entradas, simular las 2^9 posibilidades que tenemos para generar la tabla de verdad del sumador completa no es viable. Lo que he hecho es escoger una serie de señales que demuestren que el sumador funciona correctamente y da las señales de salida que debería.

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY sumador_paralelo_tb IS
END sumador_paralelo_tb;

ARCHITECTURE behavior OF sumador_paralelo_tb IS
    COMPONENT sumador_paralelo
    PORT(
        A : IN    std_logic_vector(3 downto 0);
        B : IN    std_logic_vector(3 downto 0);
        Cin: IN    std_logic;
        Cout : OUT std_logic;
        S : OUT    std_logic_vector(3 downto 0)
    );
    END COMPONENT;

    signal A : std_logic_vector(3 downto 0) := (others => '0');
    signal B : std_logic_vector(3 downto 0) := (others => '0');
    signal Cin: std_logic := '0';

    signal Cout : std_logic;
    signal S : std_logic_vector(3 downto 0);

BEGIN
    uut: sumador_paralelo PORT MAP (
        A => A,
        B => B,
        Cin => Cin,
        Cout => Cout,
        S => S
    );
    A <= "0001" after 100 ns, "0010" after 200 ns, "0011" after 300 ns, "0101"
after 400 ns, "1110" after 500 ns, "0001" after 600 ns, "0010" after 700 ns,
"0011" after 800 ns, "0101" after 900 ns, "1110" after 1000 ns;
    B <= "0001" after 100 ns, "0010" after 200 ns, "0011" after 300 ns, "0101"
after 400 ns, "1110" after 500 ns, "0001" after 600 ns, "0010" after 700 ns,
"0011" after 800 ns, "0101" after 900 ns, "1110" after 1000 ns;
    Cin <= '1' after 500 ns, '0' after 1000 ns;
END;
```

Al no tratarse de una lógica secuencial no se define señal de reloj en el Test bench. Cabe destacar que la lista de valores que se asigna en este documento será común a todas las implementaciones, de modo que este apartado se referenciará de aquí en adelante.

1.3. ISIM Simulation

Si se simula la salida para la lista de valores del test bench se obtiene el siguiente diagrama. En él, se observa que la suma y acarreo es correcto para todos los valores de modo que casi con total seguridad puede asumirse que no hay errores en la implementación.

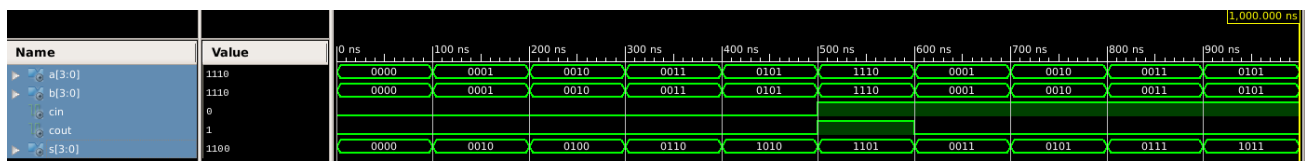


Figura 1: Simulación de un sumador de 4 bits.

1.4. Synthesize & Schematic

Desde la herramienta View RTL Schematic podemos ver como implementa el programa a partir de puertas lógicas. Desde una vista a alto nivel del circuito podemos observar las and y or que se programaron en el el archivo .vhd.

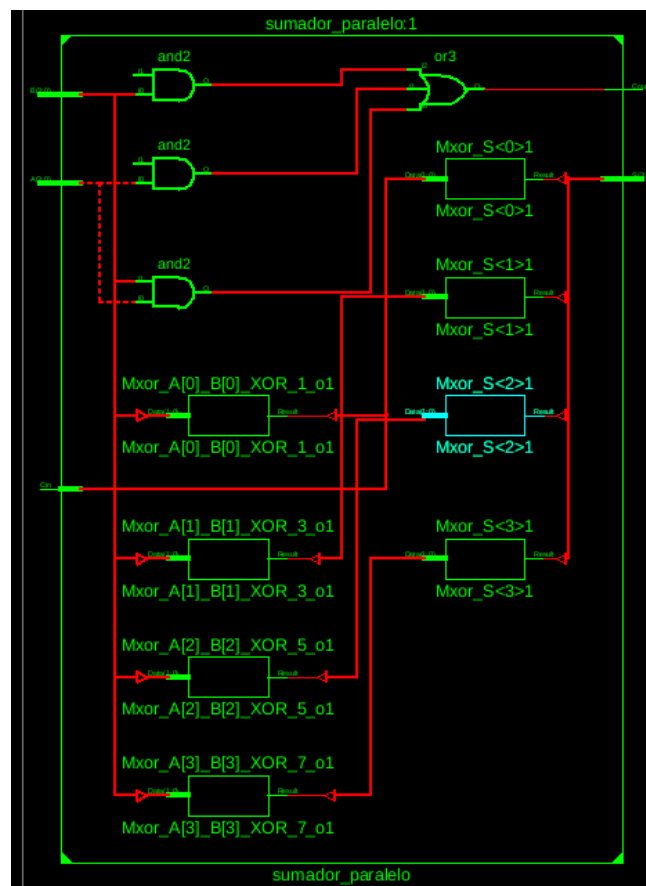


Figura 2: Esquemático de un sumador de 4 bits.

2. Utilizando descripción de comportamiento

Otra versión para el sumador de 4 bits puede elaborarse a partir de una descripción del comportamiento general del sumador. Esto ahorra la necesidad de definir los 4 sumadores completos junto con sus acarreos en el código. Generaremos la funcionalidad del sistema a partir de las señales.

Se definen las señales de entrada y salida binarias del mismo modo que al inicio, lo único que cambia es la declaración de una señal auxiliar en la arquitectura. Esta señal, de 5 bits, contendrá la suma de los 4 bits de A y B y el acarreo de entrada.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity sumador_paralelo_v2 is
    Port ( A : in  STD_LOGIC_VECTOR (3 downto 0);
          B : in  STD_LOGIC_VECTOR (3 downto 0);
          Cin: in  STD_LOGIC;
          Cout : out  STD_LOGIC;
          S : out  STD_LOGIC_VECTOR (3 downto 0));
end sumador_paralelo_v2;
```

Donde A y B son las entradas de 4 bits y Cin el acarreo de entrada que pudiera tener el sumador si se concatenase con otro para generar un sumador de n-bits. La salida S también será un vector de 4 bits.

2.1. VHDL Module

La arquitectura de la entidad **sumador paralelo v2** se hará a partir de la operación aritmética de la suma. Para sumar dos señales es conveniente que todas tengan los mismos bits, para conseguirlo se utiliza el comando &. Se define además una señal auxiliar de 5 bits que incluirá Cin y la suma de los 4 bits.

```
architecture Behavioral of sumador_paralelo_v2 is
    signal sum: std_logic_vector(4 downto 0) := (others => '0');
begin
    sum <= ('0' & A) + ('0' & B) + ("0000" & Cin);
    S <= sum(3 downto 0);
    Cout <= sum(4);
end Behavioral;
```

2.2. VHDL Test Bench

En el archivo del test bench se especifican los valores que le daremos a las señales de entrada durante la simulación. Las señales seguirán las mismas asignaciones que durante la simulación que antes. Hay que tener en cuenta que en el código deberá cambiarse *sumador paralelo* por *sumador paralelo v2* para que funcione. Al tener las mismas señales de entrada y salida el test bench no se modifica en ningún otro aspecto.

2.3. ISIM Simulation

Si se simula la salida para la lista de valores del test bench se obtiene el siguiente diagrama. En él, se observa que la suma y acarreo es correcto para todos los valores simulados. Casi con total seguridad puede asumirse que no hay errores en la implementación.

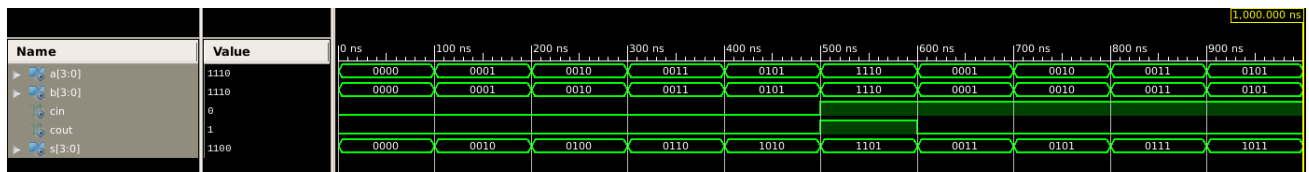


Figura 3: Simulación de un sumador de 4 bits.

2.4. Synthesize & Schematic

Desde la herramienta View RTL Schematic podemos ver como implementa el programa a partir de puertas lógicas.

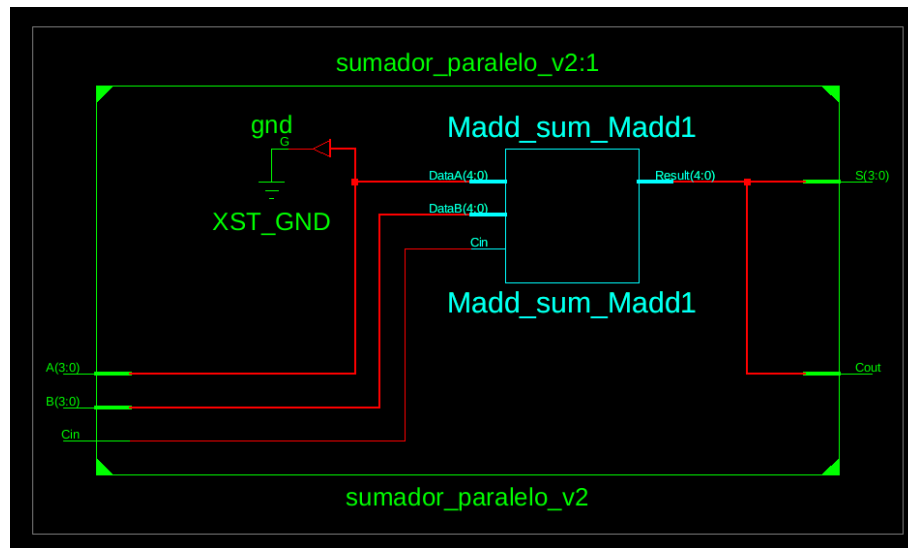


Figura 4: Esquemático de un sumador de 4 bits.

3. Utilizando descripción estructural usando sumadores completos y un semi-sumador

Otra versión para el sumador de 4 bits puede elaborarse generando las entidades de sumadores binarios en archivos independiente. Después se integrarán todos en el código del sumador de 4 bits. Un sumador de n bits, al formarse a partir de n sumadores de un bit, tiene una construcción que favorece mucho esta práctica. Al definir el sumador en una entidad, bastará con llamar a esta entidad especificando las señales de entrada y salida en el código principal sin necesidad de definir la arquitectura de nuevo.

Se definen las señales de entrada y salida binarias del mismo modo que al inicio, lo único que cambia es la declaración de una señal auxiliar en la arquitectura. Esta señal, de 5 bits, contendrá la suma de los 4 bits de A y B y el acarreo de entrada.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sumador_paralelo_v3 is
    Port ( A : in  STD_LOGIC_VECTOR (3 downto 0);
          B : in  STD_LOGIC_VECTOR (3 downto 0);
          Cout : out  STD_LOGIC;
          S : out  STD_LOGIC_VECTOR (3 downto 0));
end sumador_paralelo_v3;
```

Un half adder se codifica del siguiente modo:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Cout : out  STD_LOGIC;
        S : out  STD_LOGIC
    );
end half_adder;

architecture behavioral_ha of half_adder is
begin
    Cout <= (A and B);
    S <= A xor B;
end behavioral_ha;
```

Este sumador no tiene acarreo de entrada. Del mismo modo un sumador completo se define así:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    Port (
        A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        Cin : in  STD_LOGIC;
        Cout : out  STD_LOGIC;
        S : out  STD_LOGIC
    );
end full_adder;
```

```

    );
end full_adder;

architecture behavioral_fa of full_adder is
begin
    Cout <= (A and B) or (A and Cin) or (B and Cin);
    S <= A xor B xor Cin;
end behavioral_fa;

```

3.1. VHDL Module

En la arquitectura de la entidad **sumador paralelo v3** se llamará a estas entidades especificando el nombre de la entidad y la aquitectura que quiera implementarse.

```

architecture Struct of sumador_paralelo_v3 is
    signal C: std_logic_vector(2 downto 0) := (others => '0');
begin
    sum0 : entity work.half_adder(behavioral_ha)
        port map(A(0), B(0), C(0), S(0)); — '0' como Cin en sum0
    sum1 : entity work.full_adder(behavioral_fa)
        port map(A(1), B(1), C(0), C(1), S(1));
    sum2 : entity work.full_adder(behavioral_fa)
        port map(A(2), B(2), C(1), C(2), S(2));
    sum3 : entity work.full_adder(behavioral_fa)
        port map(A(3), B(3), C(2), Cout, S(3));
end Struct;

```

3.2. VHDL Test Bench

En el archivo del test bench se especifican los valores que le daremos a las señales de entrada durante la simulación. Las señales seguirán las mismas asignaciones que durante la simulación que antes. Hay que tener en cuenta que en el código deberá cambiarse *sumador paralelo* por *sumador paralelo v3* para que funcione. Al tener las mismas señales de entrada y salida el test bench no se modifica en ningún otro aspecto.

3.3. ISIM Simulation

Si se simula la salida para la lista de valores del test bench se obtiene el siguiente diagrama. En él, se observa que la suma y acarreo es correcto para todos los valores simulados. Casi con total seguridad puede asumirse que no hay errores en la implementación.

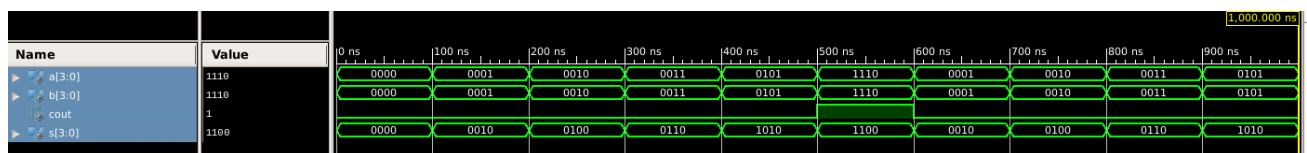


Figura 5: Simulación de un sumador de 4 bits.

3.4. Synthesize & Schematic

Desde la herramienta View RTL Svhematic podemos ver como implementa el programa a partir de puertas lógicas. Desde una vista a alto nivel del cicuito podemos observar las and y or que se programaron en el el archivo .vhd. Si se hace zoom a uno de los sumadores completos puede verse más en detalle la implementación que ha realizado el programa.

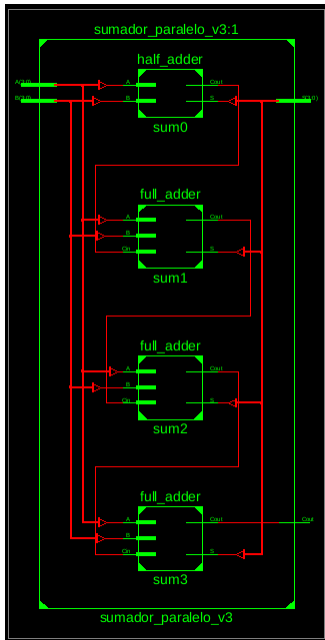


Figura 6: Esquemático de un sumador de 4 bits.

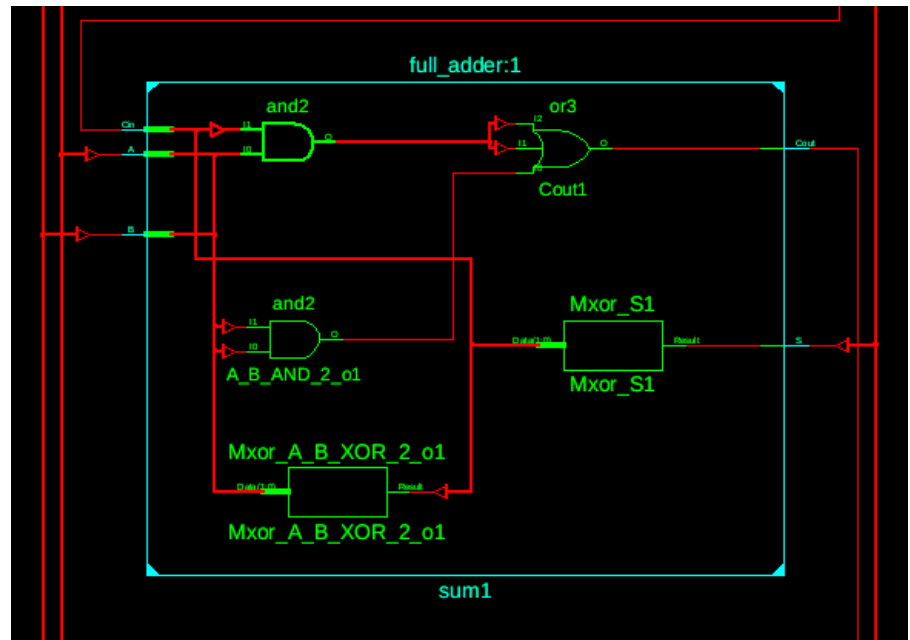


Figura 7: Esquemático de un sumador completo de 1 bit

4. Utilizando descripción estructural usando sumadores completos y un semi-sumador

Si ahora simplemente modificamos el código main de la entidad **sumador paralelo v3** implementando los módulos FA y HA mediante un for generate podremos ahorrarnos algunas líneas de código. La peculiaridad de esta técnica es, que en este código solo se especifican las conexiones que tendrán los módulos, no su arquitectura. Es por esto que podremos modificar el sistema sin necesidad de tocar los módulos. Esto facilita mucho la claridad del proyecto, sobre todo si este fuera de mayor escala.

4.1. VHDL Module

En la arquitectura de la entidad **sumador paralelo v3** se llamará a estas entidades especificando el nombre de la entidad y la arquitectura que quiera implementarse. Ahora ya no se repite el mismo comando 3 veces como antes sino que se genera un bucle for del siguiente modo:

```
architecture Struct of sumador_paralelo_v3 is
    signal C: std_logic_vector(3 downto 0) := (others => '0');
begin
    half_add_inst : entity work.half_adder(behavioral_ha)
        port map(A(0), B(0), C(0), S(0));

    gen: for i in 1 to 3 generate
        full_add_inst : entity work.full_adder(behavioral_fa)
            port map(A(i), B(i), C(i-1), C(i), S(i));
    end generate;
    Cout <= C(3);
end Struct;
```

La entidad full-add-inst ahora será un array de 3 elementos por así decirlo. Más adelante se observará mejor en los nombres del Schematic.

4.2. VHDL Test Bench

En el archivo del test bench se especifican los valores que le daremos a las señales de entrada durante la simulación. Las señales seguirán las mismas asignaciones que durante la simulación que antes. Hay que tener en cuenta que en el código deberá cambiarse *sumador paralelo* por *sumador paralelo v3* para que funcione. Al tener las mismas señales de entrada y salida el test bench no se modifica en ningún otro aspecto.

4.3. ISIM Simulation

Si se simula la salida para la lista de valores del test bench se obtiene el siguiente diagrama. En él, se observa que la suma y acarreo es correcto para todos los valores simulados. Casi con total seguridad puede asumirse que no hay errores en la implementación.

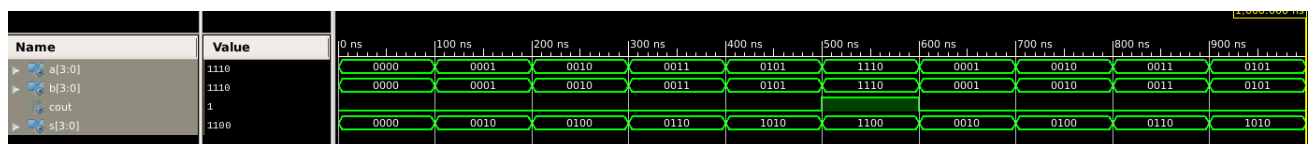


Figura 8: Simulación de un sumador de 4 bits.

4.4. Synthesize & Schematic

Desde la herramienta View RTL Svhematic podemos ver como implementa el programa a partir de puertas lógicas. Desde una vista a alto nivel del circuito podemos observar las and y or que se programaron en el el archivo .vhd. Si se hace zoom a uno de los sumadores completos puede verse más en detalle la implementación que ha realizado el programa.

Algo a destacar es que la implementación es idéntica al caso anterior, el único detalle diferente son los

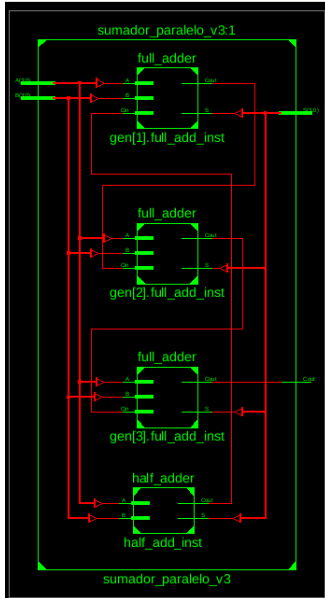


Figura 9: Esquemático de un sumador de 4 bits.

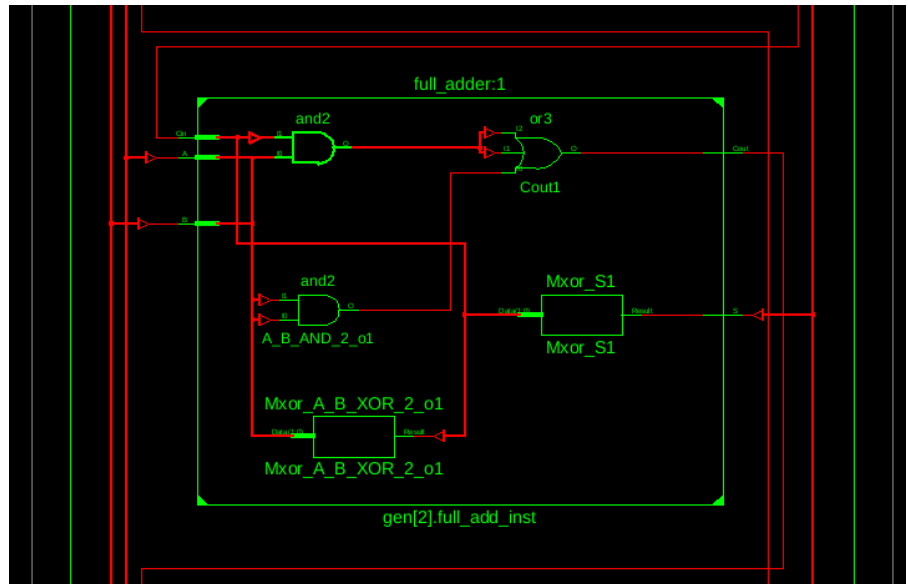


Figura 10: Esquemático de un sumador completo de 1 bit

nombres de los full-adders, que ahora vienen todos instanciados como full-add-inst.